

Refactoring Report: Application of Design Patterns

Overview

This report details the implementation of **6 Software Design Patterns** in the **AudioRent project** (React + Spring Boot). The primary objectives of this refactoring were to improve code organization, enhance reusability, ensure maintainability, and facilitate future scalability.

1. Factory Pattern (Creational)

Where it was applied: Backend - UserFactory.java

- **Before:** Object creation logic for new users (Customer and Google OAuth users) was embedded directly inside AuthService.java using inline Builders. This cluttered AuthService and tightly coupled it to the User representation.
- **After:** UserFactory handles the instantiation of User entities with default fields pre-filled based on the user type.
- **Justification & Improvement:** Reusability and separation of concerns. AuthService delegates the creation logic to the factory, improving testability and code cleanliness. If new user roles are added, they can be seamlessly added to the factory without modifying AuthService.

2. Singleton Pattern (Creational)

Where it was applied: Frontend - ApiService.ts

- **Before:** Axios instances were created and exported directly from api.ts. If multiple files imported api.ts, and modifications to interceptors occurred, synchronization could fail. It didn't enforce a strict singleton class approach.
- **After:** ApiService was introduced, defining a getInstance() method and making the constructor private, guaranteeing only one Axios configuration exists across the entire frontend application.
- **Justification & Improvement:** Provides a single, globally accessible instance of the API client and guarantees safe state management for API headers and tokens.

3. Facade Pattern (Structural)

Where it was applied: Backend - AuthFacade.java

- **Before:** AuthController was dependent on multiple services: AuthService and JwtUtils. The controller manually handled login, fetching user details, and then generating the JWT token.
- **After:** AuthFacade encapsulates AuthService and JwtUtils. The AuthController now

invokes single, high-level methods on the AuthFacade (e.g., `authFacade.processGoogleLogin()`), receiving a complete response map.

- **Justification & Improvement:** Simplifies the controller by hiding the complex orchestration of user authentication and JWT generation behind a single clean facade.

4. Adapter Pattern (Structural)

Where it was applied: Backend - `GoogleAuthAdapter.java` implementing `OAuthProvider` interface.

- **Before:** The `GoogleIdTokenVerifier` (a third-party Google SDK class) was tightly coupled directly inside `AuthService`.
- **After:** A generic `OAuthProvider` interface was introduced, with `GoogleAuthAdapter` containing the Google-specific verification logic. `AuthService` relies only on the interface.
- **Justification & Improvement:** Protects the domain logic (`AuthService`) from changes in external libraries. We can now easily add `AppleAuthAdapter` or `FacebookAuthAdapter` in the future without modifying `AuthService`.

5. Strategy Pattern (Behavioral)

Where it was applied: Backend - `UserValidationStrategy` and `EmailValidationStrategy`.

- **Before:** Basic hardcoded validation statements resided at the top of the registration method inside `AuthService`.
- **After:** `UserValidationStrategy` defines a contract for validation rules. `EmailValidationStrategy` implements it to validate the user object dynamically.
- **Justification & Improvement:** Ensures the Open/Closed Principle. We can introduce new strategies like `PasswordStrengthValidationStrategy` without modifying existing registration code.

6. Observer Pattern (Behavioral)

Where it was applied: Backend - `UserRegisteredEvent` and `EmailNotificationListener`.

- **Before:** No post-registration workflows existed. If we wanted to send a welcome email, we would have to add email-sending logic directly at the end of the register method in `AuthService`.
- **After:** Used Spring's `ApplicationEventPublisher` to broadcast a `UserRegisteredEvent`. The `EmailNotificationListener` catches the event in a decoupled manner.
- **Justification & Improvement:** Decouples the core transaction (user registration) from side effects (like notifications). `AuthService` does not need to know *what* happens after registration; it only announces that it occurred.